

Evidence 3 - Parallelizing a FFN-Layer

Moritz Trieu

May 2026

1 Introduction

The problem chosen was the calculation of a linear matrix combination $\mu(AX + b)$, where $A \in N \times M, x \in N \times M$ and $b \in N$. This computation is the basis of modern deep learning as it is the attribute computation in a feed-forwards layer as can be found in literature, for example in [Goodfellow et al., 2016]. In application N and M can be very large for example $N, M > 10,000$. In this case calculating the matrix product can be very demanding for a single core or thread. To speed up the process, the paradigm of parallelization was chosen. This splits up the operations onto different cores that can calculate different elements of the solution at the same time. In an optimal world this would reduce the computation time by the number of available cores $T_{\text{parallel}} = \frac{T_{\text{single}}}{N_{\text{cores}}}$. This is especially helpful if a large number of cores is available such as in GPUs.

2 Model

Splitting matrix addition into multiple parts is straight forward. Each thread can attach to one unique cell in the solution matrix, add the corresponding cells of the input matrices and write the solution into the corresponding matrix. Therefore, each cell can be extracted into an individual task. Thus the main challenge is the index linearization from the host matrix to the device vector.

Matrix multiplication on the other hand becomes a bit more complicated. Calculating the i_{th} component of the output vector is defined as $c_i = \sum_{j=0}^{M-1} a_{ij} \cdot x_{ji}$. Splitting up the task is for each matrix element of the output vector leaves each task with M operations. While easiest to implement it limits the distribution of work and each tasks workload scales with the size of the problem.

2.1 Linearization of Indices

When copying matrices to the GPU memory, they get linearized row by row into an array as can be seen in Fig. 1. The index mapping from matrix to vector can be calculated by $i_{vec} = i_{mat} \cdot N + j_{mat}$. From a visual standpoint this means a column index gets offset by the number of full rows prior.

$$\begin{bmatrix} a_{11} & \dots & a_{1N} \\ a_{21} & \dots & a_{2N} \\ \vdots & \ddots & \vdots \\ a_{N1} & \dots & a_{NN} \end{bmatrix} \rightarrow [a_{11} \quad \dots \quad a_{1N} \quad a_{21} \quad \dots \quad a_{2N} \quad \dots \quad a_{N1} \quad \dots \quad a_{NN}]$$

Figure 1: Linearizing a matrix by appending each row onto the previous.

In the case of CUDA, each core also uses threads. Still, it might hold true, that the problem size bigger than the number of available threads. In this case we need a way to assign tasks without the possibility of threads overlapping tasks. To this end, we first need the global thread ID which can be calculated using $i_{\text{global}} = i_{\text{thread}} + i_{\text{block}} \cdot d_{\text{block}}$. The dimension of a block is the number of threads per block. So the local ID is comprised of the global amount of threads per core before with the local thread ID. To assign to this thread a new task, we can add the number of globally available threads, so we shift the global position by the entire size of the thread pool. This can be done until the size of the problem is reached as is visible in Lst. 1.

Listing 1: Assignment of threads when the problem size is bigger than the available pool.

```
int idx = threadIdx.x + blockIdx.x * blockDim.x;

int i = idx;
while (i < vec_size) {
    ...;
    i += blockDim.x * gridDim.x;
}
```

Each block is limited to a number of threads as specified in [NVIDIA, 2026]. This means one has to increase the number of blocks. Since each block has an integer ID, the number of IDs are limited. In big problems with a large amount of cores, where these IDs are not sufficient, one can make the threads 2 dimensional, squaring the number of available IDs. In this case, the global thread ID can be calculated by first calculating the global row and column index from the thread and block IDs which then in turn can be used to calculate the global thread ID as can be seen in in Lst. 2

Listing 2: Computing the global thread ID in the case of 2D blocks.

```
int row = threadIdx.x + blockIdx.x * blockDim.x;
int column = threadIdx.y + blockIdx.y * blockDim.y;

int thread_id = row * M + column;
while (thread_id < N*N) {
    ...;
```

```

        thread_id += blockDim.x * gridDim.x;
    }

```

3 Implementation

The C++ implementation can be found in the file `Evidence_3_Matmul_parallel.ipynb` (TC2037-Evidence-3/blob/main/Evidence_3_Matmul_parallel.ipynb).

The element-wise matrix multiplication implementation can be found in Lst.

3. An important observation is the linearization of indices to account for the different directions of index movement.

Listing 3: Element-wise matrix multiplication.

```

float sum = 0;
for (int j= 0; j<M; j++) {
    sum += a[i * M + j] * x[i + j * N];
}

```

Another important aspect is variable definition and memory allocation on both device and host, as well as the setup of 2 dimensional blocks which can be seen in Lst. 4.

Listing 4: Setup of 2D blocks using dim3.

```

dim3 blocks_multi((N / THREADS) / 2, (N / THREADS) / 2);
dim3 threads_multi(THREADS, THREADS);
matmult << <blocks_multi, threads_multi >> >(d_a, d_b, d_x, d_c, N, M);

```

4 Test

To test the algorithm two different test are run.

The first test uses a small example of $N = 4$, $M = 2$ with an anisotropic number of blocks and smaller number of threads than tasks. This small example serves the purpose of testing (visually) if the result is correct and is testing the equation

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (1)$$

The second test measures the speed difference between single-core and multi-core for different problem sizes. The results can be seen in Fig. ??

5 Analysis

6 Conclusion

References

- [Goodfellow et al., 2016] Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. <http://www.deeplearningbook.org>.
- [NVIDIA, 2026] NVIDIA (2026). Cuda c++ programming guide (legacy) - cuda c++ programming guide_2026b.